

TREE

What is a Tree?

A **Tree** is a non-linear hierarchical data structure that consists of nodes connected by edges.

It is used to represent hierarchical relationships between elements.

Unlike arrays, stacks, and queues, trees do not store elements sequentially.

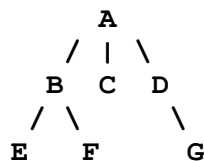
A tree starts with a special node called the **Root Node** and branches into subtrees.

Real-Life Examples of Trees

- Family hierarchy
- Organization structure
- File system in computers
- HTML/XML document structure

Basic Terminologies of Tree

Consider the following tree:



1. Node

Each element in a tree is called a **Node**.

Example:

- A, B, C, D, E, F, G are nodes.

2. Root Node

The topmost node of the tree is called the **Root Node**.

Example:

- A is the root node.

3. Parent Node

A node that has child nodes is called a **Parent Node**.

Example:

- A is parent of B, C, D
- B is parent of E and F

4. Child Node

Nodes that descend from another node are called **Child Nodes**.

Example:

- B, C, D are children of A

5. Leaf Node

Nodes that do not have any children are called **Leaf Nodes**.

Example:

- E, F, C, G are leaf nodes.

6. Sibling Nodes

Nodes having the same parent are called **Siblings**.

Example:

- B, C, D are siblings.

7. Edge

The connection between two nodes is called an **Edge**.

Example:

- $A \rightarrow B$
- $B \rightarrow E$

8. Degree of Node

The number of children of a node is called its **Degree**.

Example:

- Degree of A = 3
- Degree of B = 2

9. Height of Tree

The maximum number of edges from root to leaf is called the **Height** of the tree.

10. Level of Node

The position of a node in the hierarchy is called its **Level**.

- Root node is at Level 0.

11. Subtree

A tree formed by a node and its descendants is called a **Subtree**.

Properties of Trees

1. A tree contains a set of connected nodes.
2. A tree with **N nodes** has **N-1 edges**.
3. There is exactly one path between any two nodes.
4. Trees do not contain cycles.
5. The root node has no parent.
6. Every child has exactly one parent.

Binary Tree

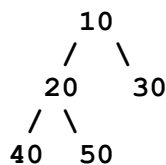
What is a Binary Tree?

A **Binary Tree** is a tree in which each node can have at most **two children**.

These children are called:

- Left Child
- Right Child

Example:



Types of Binary Tree

1. Full Binary Tree

A binary tree where every node has either:

- 0 children
OR
- 2 children

Example:



2. Complete Binary Tree

All levels are completely filled except possibly the last level.

Nodes are filled from left to right.

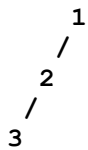
3. Perfect Binary Tree

All internal nodes have 2 children and all leaf nodes are at the same level.

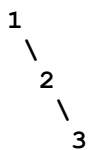
4. Skewed Binary Tree

A tree where all nodes have only one child.

Left Skewed

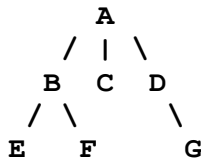


Right Skewed



Height and Degree in Trees

Consider the following tree:



1. Degree of a Node

Definition

The **Degree of a Node** is the number of children that node has.

Finding Degree of Nodes

Node	Children	Degree
A	B, C, D	3
B	E, F	2
C	—	0
D	G	1
E	—	0
F	—	0
G	—	0

Important Point

- Leaf nodes always have degree 0
- Degree depends on number of children

2. Degree of a Tree

Definition

The **Degree of a Tree** is the maximum degree among all nodes in the tree.

Example

From the above tree:

- Degree of A = 3
- Degree of B = 2
- Degree of D = 1

Maximum degree = 3

Therefore:

Degree of Tree = 3

3. Height of a Tree

Definition

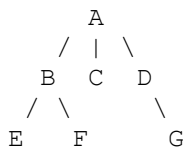
The **Height of a Tree** is the number of edges in the longest path from the root node to a leaf node.

OR

It is the maximum depth of the tree.

Finding Height Step-by-Step

Consider the same tree:



Paths from Root to Leaf

Path 1

A → B → E

Edges = 2

Path 2

$A \rightarrow B \rightarrow F$

Edges = 2

Path 3

$A \rightarrow C$

Edges = 1

Path 4

$A \rightarrow D \rightarrow G$

Edges = 2

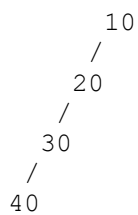
Longest Path

Maximum edges = 2

Therefore:

Height of Tree = 2

Another Example



Longest path:

$10 \rightarrow 20 \rightarrow 30 \rightarrow 40$

Number of edges = 3

Height = 3

Summary Table

Concept	Meaning
Degree of Node	Number of children of a node
Degree of Tree	Maximum degree among all nodes
Height of Tree	Longest path from root to leaf

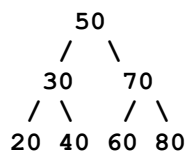
Binary Search Tree (BST)

What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a binary tree that follows these rules:

- Left subtree contains smaller values
- Right subtree contains larger values
- Both left and right subtrees are also BSTs

Example:



Properties of BST

1. Left child < Parent
2. Right child > Parent
3. Searching becomes efficient

Construct a BST from the Following Numbers

50, 30, 70, 20, 40, 60, 80

Step-by-Step BST Formation

Insert 50

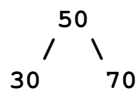
50

Insert 30

$30 < 50 \rightarrow$ goes to left

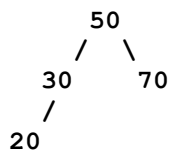
**Insert 70**

$70 > 50 \rightarrow$ goes to right

**Insert 20**

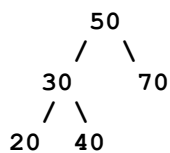
$20 < 50$

$20 < 30 \rightarrow$ left of 30

**Insert 40**

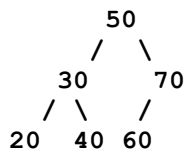
$40 < 50$

$40 > 30 \rightarrow$ right of 30

**Insert 60**

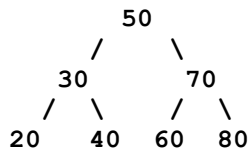
$60 > 50$

$60 < 70 \rightarrow$ left of 70

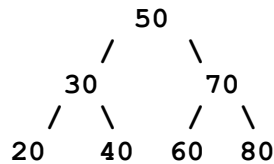
**Insert 80**

$80 > 50$

$80 > 70 \rightarrow$ right of 70



Final BST



Tree Traversals

Traversal means visiting all nodes of the tree in a specific order.

There are mainly 3 Depth First Traversals:

1. Preorder Traversal
2. Inorder Traversal
3. Postorder Traversal

1. Preorder Traversal

Definition

In **Preorder Traversal**, nodes are visited in the following order:

Root → Left → Right

Steps for Above Tree

- Visit Root → 50
- Traverse Left Subtree → 30 20 40
- Traverse Right Subtree → 70 60 80

Preorder Answer

50 30 20 40 70 60 80

2. Inorder Traversal

Definition

In **Inorder Traversal**, nodes are visited in the following order:

Left → Root → Right

Steps for Above Tree

- Left Subtree → 20 30 40
- Root → 50
- Right Subtree → 60 70 80

Inorder Answer

20 30 40 50 60 70 80

Important Point

Inorder traversal of BST always gives elements in **sorted order**.

3. Postorder Traversal

Definition

In **Postorder Traversal**, nodes are visited in the following order:

Left → Right → Root

Steps for Above Tree

- Left Subtree → 20 40 30
- Right Subtree → 60 80 70
- Root → 50

Postorder Answer

20 40 30 60 80 70 50

Easy Trick

Traversal	Remember As
Preorder	Root first
Inorder	Root middle
Postorder	Root last

Construct a BST using:

1. 50, 30, 70, 20, 40, 60, 80
2. 40, 20, 60, 10, 30, 50, 70
3. 60, 40, 80, 20, 50, 70, 90